

PILHA

Uma **pilha** (= *stack*) é uma lista de dados que só aceita remoção do último elemento e só aceita inserção após o último elemento. Portanto, o último a entrar é o primeiro a sair (política Last-In-First-Out).

Várias aplicações reais são implementadas usando pilhas, por exemplo, em editores de texto, as operações “undo” usam uma pilha para armazenar as últimas alterações realizadas.

A pilha é uma estrutura de dados que tem uma única extremidade, pela qual, os dados são inseridos e removidos. Esta extremidade é denominada topo da pilha. Embora, conceitualmente, a pilha seja uma estrutura de dados dinâmica (aumenta e diminui de maneira imprevisível durante a execução do programa), basicamente, existem duas estratégias para implementar uma pilha: implementação estática e dinâmica. Na implementação estática, utilizamos um vetor para armazenar os dados na pilha. Já na implementação dinâmica, utilizamos ponteiros. Vamos abordar a implementação dinâmica.

Na implementação dinâmica, a pilha é formada por uma sequência (lista) de nós, o último nó empilhado é sempre o topo da pilha. Cada nó é definido pelo valor que ele armazena e por um ponteiro que aponta para o próximo elemento da pilha.

Por exemplo, suponha que você queira criar uma pilha de números inteiros maiores que 0. Um nó desta pilha terá a seguinte estrutura:

```
struct no {  
    int dado; // neste campo, é armazenado o valor do nó, neste caso, um valor  
    inteiro  
    struct no *prox; // este campo é um ponteiro, ou seja, ele guarda o endereço  
    onde o próximo nó da pilha está alocado, quando este nó não tem próximo, o ponteiro  
    fica null.  
}
```

Esta é a estrutura do nó, temos que definir a estrutura da nossa pilha. O que precisamos saber é o nó que é o topo da pilha. Desta forma, nossa pilha será representada por um ponteiro que vai apontar para o nó que é o topo da pilha. Neste caso, a pilha pode ser definida da seguinte forma:

```
struct pilha{  
    struct no *topo; //topo guarda o endereço do nó que é o topo da pilha  
}
```

Ao manipular uma pilha como estrutura de dados, deve-se considerar as operações que poderão ser realizadas. Neste contexto, podemos citar as seguintes operações:

- iniciaPilha ou criaPilha
- pilhaVazia: verifica se a pilha está vazia
- empilha: inclui um elemento no topo da pilha
- desempilha: retira o elemento do topo da pilha (se a pilha não estiver vazia)

Veja que, seguindo o conceito do TAD pilha, nós não podemos acessar todos os nós da pilha, somente o topo. Suponha que você tenha empilhado três números: primeiro o 4, depois o 8 e, por último, o 5. Na pilha, os nós ficarão empilhados como na Figura 1.

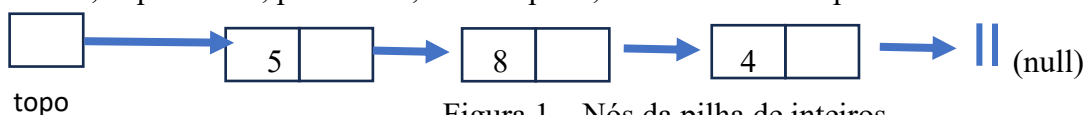


Figura 1 – Nós da pilha de inteiros

O topo da pilha aponta para o nó cujo valor é 5 (último nó inserido na pilha e o primeiro nó que poderá ser removido). A rigor, não é possível saber quem é o próximo nó depois do 5 porque, na pilha, só conseguimos acessar o topo. Se for necessário saber o próximo nó, é necessário desempilhar o topo.

Contudo, além destas operações válidas, vamos definir uma operação para facilitar a validação das nossas operações básicas. A operação deverá retornar o valor do elemento que está no topo, mas sem desempilhar:

- top: retorna o valor do elemento do topo da pilha sem retirá-lo

Vamos considerar uma aplicação que utilize uma pilha para manipular elementos inteiros maiores que 0. Considere o seguinte objetivo: na função principal, você deve perguntar ao usuário quantos elementos serão inseridos (empilhados), empilhar esses elementos e depois desempilhar e imprimir os elementos da pilha.

Uma alternativa para implementação do tdpilha está descrita a seguir:

```
typedef struct tno;  
typedef struct pilha tpilha;  
tpilha *criapilha(); //aloca memoria para pilha e inicializa o topo da pilha para null  
int pilhavazia(tpilha *p); //retorna 1 se a pilha estiver vazia, 0 do contrário  
int empilha (tpilha *p, int dado); //retorna 1, se foi possível empilhar, 0 caso contrário  
int topo(tpilha *p); //retorna o valor do topo se a pilha nao estiver vazia, -1, caso contrário  
int desempilha (tpilha *p); //desempilha o topo se a pilha nao estiver vazia, -1, caso contrário
```

```

#include<stdlib.h>
#include "tadpilha.h"
struct no {
    int dado;
    struct no *prox;};
struct pilha{ tno *topo;};
tpilha *criapilha(){
    tpilha *p;
    p = (tpilha *)malloc(sizeof(tpilha));
    if (p) p->topo = NULL;
    return p;}
int pilhavazia(tpilha *p){
    if (!p->topo)
        return 1;
    else return 0;}
int empilha (tpilha *p, int dado){
    tno *no;
    no = (tno *)malloc(sizeof(tno));
    if (!no) {
        return 0;}
    else {
        no->dado = dado;
        no->prox = p->topo;
        p->topo = no;
        return 1;}}
int topo (tpilha *p){
    if(pilhavazia(p)) return -1;
    else return p->topo->dado;}
int desempilha (tpilha *p){
    int dado;
    tno *no;
    if(pilhavazia(p)) return -1;
    else {
        dado = p->topo->dado;
        no = p->topo;
        p->topo = p->topo->prox;
        free(no);
        return dado;}}

```

```
#include "tadpilha.h"
#include<stdio.h>
int main(){
    tpilha *p;
    int cont, num;
    printf("Quantos numeros voce quer empilhar?");
    scanf("%d",&cont);
    p = criapilha();
    if (!p) return 1;
    for (int i=0;i<cont;i++){
        printf("Digite o numero: ");
        scanf("%d", &num);
        if (!empilha(p,num)){
            printf("Não tem mais espaço!");
            return 1;
        }
    }
    printf("\nDesempilhando:\n");
    while (!pilhavazia(p)){
        printf("%d\n", desempilha(p));
    }
    return 0;
}
```